

User Story

"As an authenticated user, I want to manage my daily tasks through a web system, allowing me to create, view, edit, and delete tasks, with the assurance that only I have access to my own tasks."

Acceptance Criteria:

- User can register and log in
- Session protected with JWT
- Each task contains: title, description, status, and due date
- Users can only view and manage their own tasks
- Responsive and functional interface

I used IDE Qoder with Deepseek for been my AI Tool in this project.

I started from the backend.

Firts prompt

" You are a senior Software Architect specialized in C#/.NET, REST APIs, Clean Code, and pragmatic enterprise architecture.

Your goal is to design solutions that are: simple,robust ,appropriately scalable, easy to maintain, free from overengineering .

Always prioritize: code clarity, readability, pragmatic architecture, low coupling, high cohesion, easy debugging, practical solutions commonly used in real-world enterprise systems.

Avoid unnecessary patterns and abstractions.

Do not turn a simple application into an overly complex architecture.

Create a simple, robust, production-ready backend application using: ONET 8 or higher, ASP.NET Core Web API, Pure ADO.NET, SQL Server or PostgreSQL, JWT Authentication, Native ASP.NET Core Dependency Injection.

Goal

The application must prioritize: simplicity, clarity, low coupling, maintainability, good performance, predictable code, pragmatic architecture.

Avoid any form of overengineering.

Mandatory Requirements

Use Clean Architecture, Use async/await properly, Use native dependency injection, Implement JWT authentication, Validate input data, Implement global exception handling, Use ILogger for basic logging, Use appsettings.json for configuration, Use parameterized SQL queries, Return proper HTTP responses, Create simple and consistent RESTful endpoints, Separate DTOs from domain entities, Keep the codebase clean and organized.

Data Access

Use exclusively pure ADO.NET: SqlConnection / NpgsqlConnection, SqlCommand / NpgsqlCommand, SqlDataReader.

SQL queries must be: explicit, simple, organized, easy to maintain.

Avoid generic layers and excessive abstractions.

Do not use: Entity Framework,Dapper, MediatR, CQRS, Event Sourcing, Microservices, Unnecessary Unit of Work pattern, Excessive design patterns,“Enterprise architecture” without real need

Security

Implement: JWT Bearer Authentication, secure password hashing, SQL Injection protection, simple role-based authorization if needed.

Expected Output

Generate: complete project structure, folder organization, functional CRUD examples, working JWT authentication, real endpoint examples, real SQL query examples, local setup instructions.

The solution should look like a real professional backend used in production environments while remaining simple, direct, reliable, and easy to evolve. “

When i submit this prompt, the AI ask me if i want to build a plan before the implementation for been more acertive and i agree. So the first result is a file using markdown with a preview of a execution plan.

```
1. Domain & Entities

The application manages Products organized by Categories, with Users for authentication. A simple, realistic enterprise domain.

Database Tables (PostgreSQL):

• users (id SERIAL, username, email, password_hash, role, created_at, updated_at)
• categories (id SERIAL, name, description, created_at, updated_at)
• products (id SERIAL, name, description, price, stock_quantity, category_id FK, created_at, updated_at)

All queries are parameterized to prevent SQL injection.
```

This screenshot is part of the generated file showing entities created only for the initial product setup, so I provided which entities would exist and the purpose of the API by using this prompt:

”The API represents a simple task management system.

The system must support: Create tasks ,Read tasks ,Update tasks ,Delete tasks

Each task must contain: Id,Title ,Description,Status,DueDate,UserId

Tasks must be associated with a user. Assume a basic User model already exists. Do adjustments of the plan for fit in this new scenario. “

Thats was teh result of plan:

1. Domain & Entities

A simple **Task Management System** where authenticated users can create, read, update, and delete their tasks. Each task is associated with a user.

A basic **User** model is already assumed (used for authentication and task ownership).

Database Tables (PostgreSQL):

- `users` (id SERIAL, username, email, password_hash, role, created_at, updated_at)
- `tasks` (id SERIAL, title, description, status, due_date, user_id FK, created_at, updated_at)

```
backend/
├── TechExercise.slnx
├── docker-compose.yml
├── database/
│   └── init.sql
├── TechExercise.Domain/
│   ├── TechExercise.Domain.csproj
│   └── Entities/
│       ├── User.cs
│       └── TaskItem.cs
├── TechExercise.Application/
│   ├── TechExercise.Application.csproj
│   ├── Interfaces/
│   │   ├── IAuthService.cs
│   │   ├── ITaskService.cs
│   │   ├── IUserRepository.cs
│   │   ├── ITaskRepository.cs
│   │   ├── IJwtService.cs
│   │   └── IPasswordHasher.cs
│   ├── Services/
│   │   ├── AuthService.cs
│   │   └── TaskService.cs
│   ├── DTOs/
│   │   ├── Auth/
│   │   │   ├── RegisterRequest.cs
│   │   │   ├── LoginRequest.cs
│   │   │   └── LoginResponse.cs
│   │   └── Tasks/
│   │       ├── CreateTaskRequest.cs
│   │       ├── UpdateTaskRequest.cs
│   │       └── TaskResponse.cs
│   └── Exceptions/
│       └── ConflictException.cs
├── TechExercise.Infrastructure/
└── TechExercise.Infrastructure.csproj
```

A resume of desing decisions of project including Nuget packages, Validation, Authentication and Authorization.

Show me all endpoints for the solution:

Auth (public)		
Method	Path	Description
POST	/api/auth/register	Register new user
POST	/api/auth/login	Login, returns JWT
Tasks (authenticated -- user-scoped)		
Method	Path	Description
GET	/api/tasks	List all tasks for the authenticated user
GET	/api/tasks/{id}	Get a specific task (own user only)
POST	/api/tasks	Create a new task
PUT	/api/tasks/{id}	Update a task (own user only)
DELETE	/api/tasks/{id}	Delete a task (own user only)

AI was waiting for my approve to start implement the solution, so after i review the plan i approved.

The result was a web api solution with the structure correct, so i started to review and find some parts to improve, remove unnecessary logic, make more simple and clean. I found some SOLID principle violations for example:

DIP(Dependency Inversion)

All repositories depend on the concrete class of DbConnectionFactory

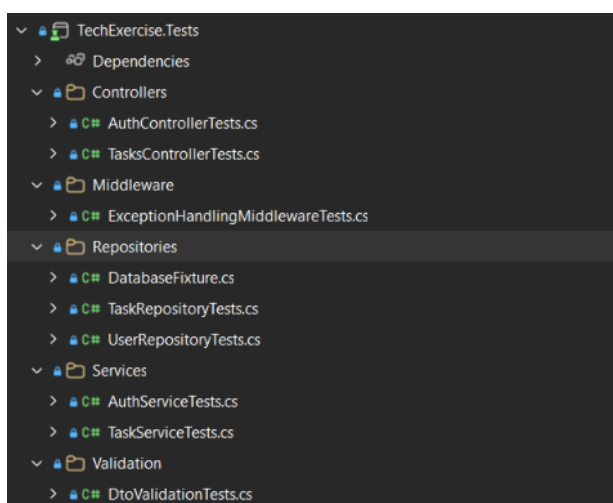
```

6 references
public class UserRepository : IUserRepository
{
    private readonly DbConnectionFactory _connectionFactory;

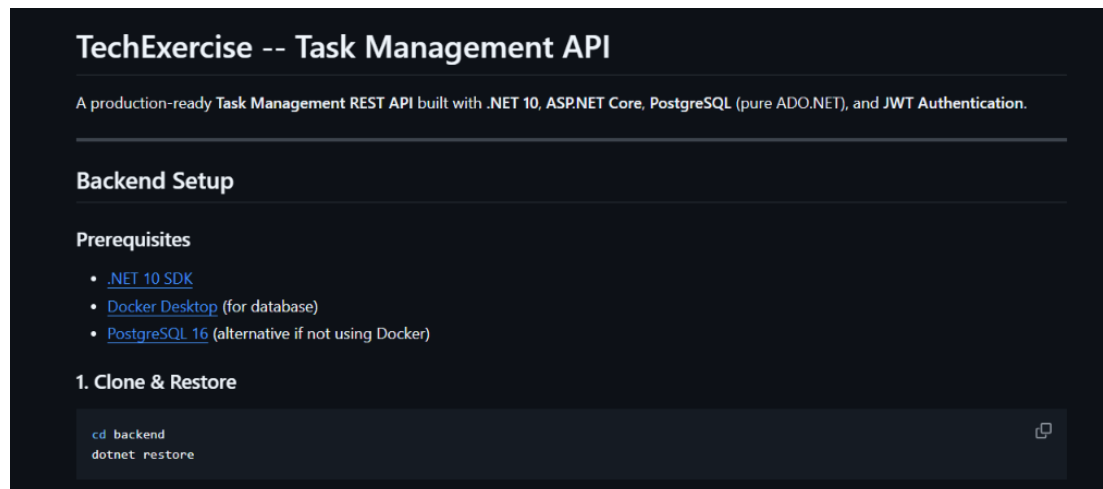
```

So i asked for AI to fix those points, then i asked to create all unit tests with this prompt:

"build unit tests using Xunit for whole application, must cover all parts os project including services, autentication, validation, controllers and data access layer. Make it in a separated project" The result was a test project created with all tests:



I verify if all unit tests was correctly and they logic. Then i asked for AI to create a Readme file with instructions of how to configure and run the application, structure of the project and information of all endpoints. Here's a screenshot of part of the Readme file:



Frontend

First prompt

"You are a Senior Frontend Engineer specialized in Angular applications.

Your task is to create a simple and clean frontend application using the latest stable version of Angular inside frontend folder.

The application is a Task Management System that communicates with an existing backend REST API and is located in the backend folder.

The goal is to create a lightweight, maintainable, responsive, and well-structured application without overengineering.

Assume that Node.js and the latest version of Angular CLI are already installed.

Main Requirements

Build a frontend application with the following features:

Authentication

- Login screen with: email/username, password
- User registration screen
- JWT authentication
- Route protection for authenticated users
- Logout functionality

Task Management

Implement CRUD operations for tasks:

- Create task
- List tasks
- Edit task
- Delete task

Each task contains: title, description, status, due date.

Technical Requirements

- Use the latest stable Angular version
- Use standalone components
- Use TypeScript strict mode
- Use Angular Material for UI components
- Use SCSS for styling
- Use Reactive Forms
- Use Angular Router
- Use Angular HttpClient
- Use Angular Signals when useful
- Keep the architecture simple and clean
- Avoid unnecessary abstractions and overengineering

UI/UX Requirements

The application must: Be fully responsive, Follow mobile-first principles, Use Angular Material components properly, Have a clean and modern interface, Include loading indicators, Include form validation messages, Include empty states when there are no tasks, Include confirmation dialogs for deletions, Include snackbar/toast notifications for success and error messages

Suggested Screens

Authentication

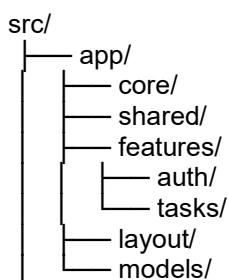
- Login Page
- Register Page

Tasks

- Task List Page
- Create/Edit Task Dialog or Page

Architecture Guidelines

Use a simple and organized structure similar to:



Requirements: Separate responsibilities clearly, Keep components reusable, Keep services focused, Avoid duplicated code, Avoid unnecessary complexity.

API Integration

The backend API already exists.

Implement: centralized API communication, environment-based API URL, JWT token interceptor, global error handling interceptor, typed request/response models, proper loading and error states.

Angular Best Practices

Follow Angular official best practices: Prefer standalone components, Use OnPush change detection, Use async pipe whenever possible, Avoid manual subscriptions when unnecessary, Use lazy loading for routes, Keep components small and focused, Avoid business logic in templates, Use strong typing, Avoid using "any"

Forms

All forms must: use Reactive Forms, include validation, show validation messages, prevent invalid submissions.

Security

Implement frontend security best practices: JWT token handling, protected routes, logout on unauthorized responses, avoid exposing sensitive data.

Testing

Generate basic unit tests for: services, components, authentication logic.

Documentation

Generate: README.md, setup instructions, project structure explanation, environment configuration example.

Important Rules

- Keep the solution simple
- Do not overengineer
- Do not use NgRx
- Do not create unnecessary layers
- Focus on readability and maintainability
- Prioritize clean code and user experience

The final result should look like a professional but simple Angular application ready for real-world usage.”

Also when i submit this prompt, the AI ask me if i want to build a plan before the implementation for been more acertive and i agree. So the first result is a file using markdown with a preview of a execution plan.

Angular Frontend for Task Management System

Architecture & API Contract Summary

Backend API Base URL: `http://localhost:50349`

Auth Endpoints:

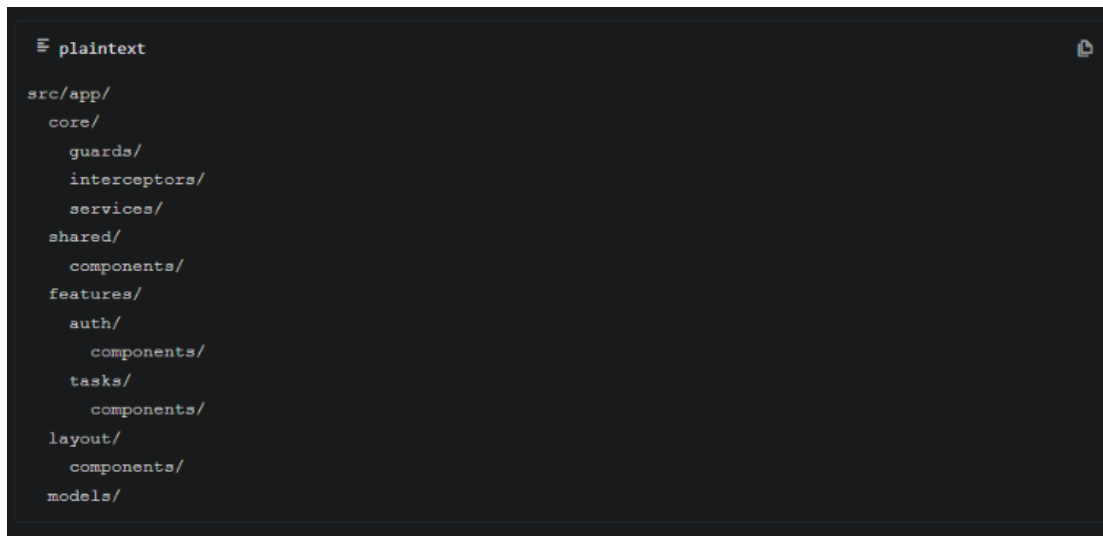
- `POST /api/Auth/register` — `Body: { username, email, password }` → `Response: { idUser, token, username, email }`
- `POST /api/Auth/login` — `Body: { email, password }` → `Response: { idUser, token, username, email }`

Task Endpoints (JWT Bearer Auth required):

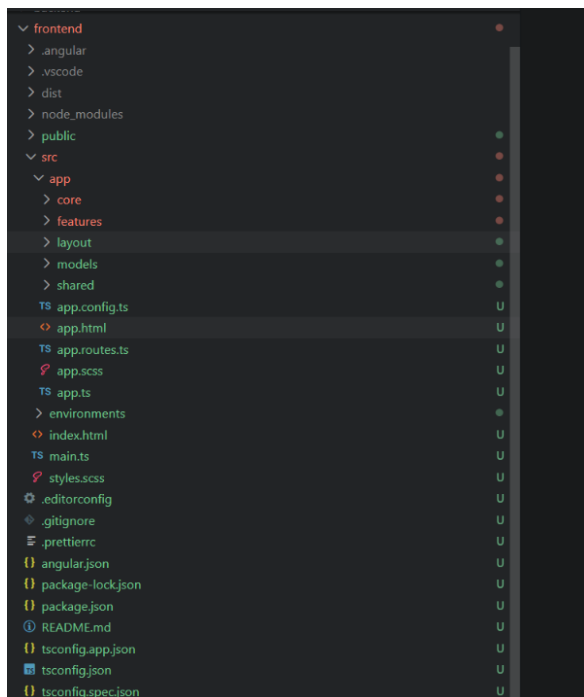
- `GET /api/Tasks` → `TaskResponse[]`
- `GET /api/Tasks/{id}` → `TaskResponse`
- `POST /api/Tasks` — `Body: { title, description, status, dueDate }` → `TaskResponse`
- `PUT /api/Tasks/{id}` — `Body: { title, description, status, dueDate }` → `TaskResponse`
- `DELETE /api/Tasks/{id}` → `204 No Content`

Task statuses: `"pending", "in_progress", "completed"`

Error response: `{ error: "message" }` with appropriate HTTP status codes.



Those screenshots are part of the plan execution generated showing endpoints to connect and structure of project. The result of plan execution was a fully implementation of frontend application:



Then i started the code review by checking if the project structure was in accordance with what I requested in the prompt, and run the application for verify if works correctly.

I found some configuration issues, that was impact the communication between front and back (CORS), so i asked for AI fix this problem. Then the system works very well. And this a final result:

Sign In

Welcome back! Enter your credentials.



Sign In

Don't have an account? [Create one](#)

My Tasks

+ New Task

Title	Status	Due Date	Actions
task 1 this is a test task	Pending	May 17, 2026	 

New Task

Title*

Enter task title

Description

Status*

Pending

Due Date

Cancel

Create

A frontend application with a clean and responsive layout integrated with backend correctly.